

Automação e Otimização de Processos de TI na UFABC: O Desenvolvimento e Implementação do ITXAutoNTI

Maria Eduarda A. B. Brito, Erik Farias Lima, Breno Barros Bianchi,
Pedro Henrique de Lima Franca, Felipe Dias C. Iglesias

¹Núcleo de Tecnologia da Informação (NTI) – Universidade Federal do ABC (UFABC)
Santo André – SP – Brasil

{maria.brito, erik.lima, b.bianchi, lima.franca, felipe.iglesias}@ufabc.edu.br

Resumo. *A gestão de infraestrutura de TI em instituições públicas demanda rigor, rastreabilidade e agilidade, mas frequentemente esbarra em processos manuais que sobrecarregam as equipes. Este artigo apresenta o desenvolvimento e a implementação do ITXAutoNTI, um sistema de automação concebido para otimizar a inserção e movimentação de patrimônios no sistema GLPI do Núcleo de Tecnologia da Informação (NTI) da UFABC. Desenvolvido em linguagem Python, o projeto consome a API REST do GLPI utilizando bibliotecas de requisições assíncronas para simular processamentos em lote. Para mitigar inconsistências na base de dados oriundas de erros humanos na digitação de localizações, o sistema integrou algoritmos de correspondência aproximada (fuzzy matching). Os resultados demonstram uma redução superior a 90% no tempo operacional de atendimento de chamados patrimoniais, a eliminação de erros de categorização e a consolidação de uma oportunidade pedagógica de capacitação em engenharia de software para a equipe de estagiários desenvolvedores.*

1. Introdução

A gestão de ativos de tecnologia da informação (TI) é um elemento central para o funcionamento eficiente de instituições de médio e grande porte. Ferramentas como o GLPI desempenham um papel fundamental no controle patrimonial, mas o uso dessas plataformas frequentemente envolve processos manuais repetitivos que impactam a produtividade e a qualidade dos dados.

No NTI da UFABC, chamados de movimentação de ativos demandavam a inserção manual de até centenas de itens por múltiplas interfaces, tornando o processo dispendioso e suscetível a erros que comprometiam a base de dados do GLPI.

Diante desse cenário, identifica-se uma lacuna na automação de processos do GLPI, especialmente para inserções e atualizações em lote. Embora o sistema ofereça recursos abrangentes, carece de mecanismos nativos para automação baseada em entradas estruturadas simples. Essa limitação evidencia a necessidade de soluções complementares para elevar a eficiência operacional preservando a integridade dos dados.

Este trabalho apresenta o desenvolvimento do ITXAutoNTI, uma ferramenta de automação integrada ao GLPI via API, projetada para processar a inserção e movimentação de múltiplos patrimônios de forma autônoma a partir de instruções padronizadas nos chamados técnicos.

2. Fundamentação e Motivação

Nas Instituições Federais de Ensino Superior (IFES), a governança de TI garante a continuidade das atividades. A adoção do *framework* ITIL preconiza o Gerenciamento de Ativos e Configuração de TI (ITAM/SACM) por meio de um repositório centralizado e atualizado para assegurar a conformidade dos serviços.

Neste contexto, a necessidade de otimização tornou-se evidente na rotina da Coordenação de Gestão e Planejamento (CGP) do NTI, responsável pelas demandas de aquisição e movimentação de equipamentos. Inicialmente, o setor adotou uma solução improvisada, uma ferramenta baseada em macro (*AutoHotkey*) para suprir demandas imediatas.

Embora reduzisse o esforço, ela apresentava limitações como a dependência de uma estação de trabalho única, a falta de suporte multiusuário e a ausência de auditoria. A alta sensibilidade a qualquer alteração na interface web do GLPI evidenciou que a abordagem não era institucionalmente sustentável.

A partir desse diagnóstico, surgiu a proposta de transformar a solução pontual em um sistema estruturado. O ITXAutoNTI foi então idealizado como uma aplicação externa, robusta e desacoplada, integrada diretamente via API REST do GLPI. Essa transição tecnológica permitiu processar operações de patrimônio em lote de forma transparente, segura e independente do posto de trabalho do usuário.

3. Metodologia

O projeto ITXAutoNTI foi concebido como uma pesquisa aplicada para mitigar o gargalo operacional NTI no sistema GLPI.

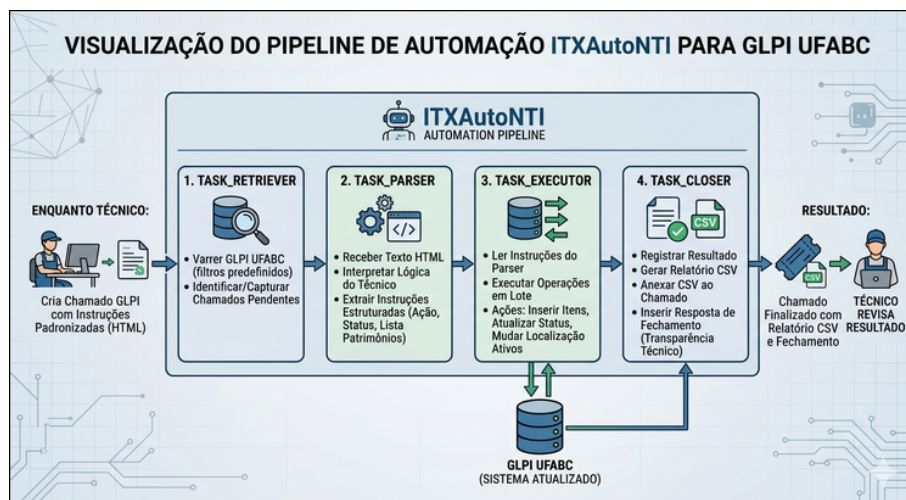
3.1. Estrutura Modular da Solução

Para viabilizar uma ferramenta robusta e de fácil manutenção, foi estruturada em componentes modulares independentes, onde cada módulo representa uma etapa do procedimento:

- **Task_Retriever:** Atua como o mecanismo de monitoramento. É responsável por varrer o sistema GLPI através de filtros predefinidos para capturar chamados pendentes que contenham as instruções específicas destinadas ao robô.
- **Task_Parser:** Executa a análise léxica do conteúdo HTML das tarefas. Ele interpreta a lógica descrita pelo técnico e extrai, de forma estruturada, as instruções de ação, novos estados (*status*/localização) e a lista de patrimônios.
- **Task_Executor:** O núcleo operacional que processa as instruções em lote. Realiza as mutações de estado via API (como inserção de itens e atualização de propriedades) diretamente nos objetos patrimoniais indicados.
- **Task_Closer:** Finaliza o ciclo de vida da automação. Este módulo encerra a tarefa original, gera um relatório CSV detalhado para auditoria e anexa o documento ao chamado, notificando o técnico responsável de forma transparente.

A Figura 1 ilustra o fluxo de dados e a integração entre os módulos desenvolvidos, demonstrando a sequência de execução desde a varredura inicial de um chamado no sistema GLPI até a sua finalização.

Figura 1. Visualização gráfica do funcionamento do pipeline do ITXAutoNTI.



Fonte: Gerada por IA (Plataforma Gemini), abril de 2026.

Essa divisão modular permitiu o desenvolvimento paralelo das funcionalidades, além de viabilizar testes unitários independentes para cada etapa do processo.

3.2. Ciclo de Desenvolvimento e Validação

Adotou-se uma abordagem iterativa orientada a entregas funcionais. O ciclo contemplou fases de validação contínua em ambiente de homologação, visando mitigar desafios comuns de integrações RESTful (como latência de resposta). Após testes isolados na fase *Alpha*, a aplicação evoluiu para a versão de produção (v1.1.0), consolidando uma arquitetura de alta disponibilidade e segurança para o ambiente institucional.

3.3. Arquitetura da Solução e Segurança

Do ponto de vista técnico, embora o ecossistema nativo do GLPI seja construído em linguagem PHP, optou-se pelo uso de Python 3.10+ na implementação da automação, fundamentando-se em dois pilares principais: a agilidade da linguagem para o consumo de APIs RESTful e o viés pedagógico inerente à iniciativa. O uso de Python gerou uma curva de aprendizado favorável aos estagiários do NTI. Essa escolha permitiu consolidar práticas como o uso de ambiente virtual, modularização e versionamento Git, transformando o problema operacional em um exercício de engenharia de software aplicada.

A arquitetura foi desenhada de modo a proteger o acesso aos dados institucionais para garantir a segurança na integração com o GLPI via API REST. O sistema utiliza um esquema de autenticação dupla: um *App-Token* (que identifica a aplicação perante a API) e um *User-Token* (que autentica as requisições em nome de um usuário de serviço com permissões restritas). Para prevenir possíveis exposições do controle de acesso, essas credenciais sensíveis não são inseridas diretamente no código-fonte, mas sim gerenciadas de forma segura através de variáveis de ambiente do sistema operacional.

3.4. Tratamento de Inconsistências na Localização

Diferentemente da ação (inserir/remover) e do *status* (ativo, em estoque, etc.), que possuem um número reduzido de opções fixas, as **localizações** dos ativos são muitas e variam

conforme o ambiente (laboratórios, salas, torres, blocos). No GLPI, não há uma lista padronizada para os técnicos escolherem, eles simplesmente digitam o nome do local em texto livre, gerando inconsistências. Um mesmo local pode ser escrito de várias formas (ex.: 'Lab Info 401' e 'L401'), gerando inconsistências na escrita que prejudicariam o funcionamento da API.

Para mitigar essa inconsistência sem exigir mudanças no fluxo de trabalho do usuário, o módulo incorporou um mecanismo analítico de **correspondência aproximada** (*fuzzy matching*), arquitetado para processamento *case-insensitive*. A inteligência compara o texto de entrada com a base de dados oficial. Caso a similaridade atinja o limiar estipulado, ocorre a conversão autônoma. Para cenários anômalos, desenvolveu-se um *fallback* que adapta a busca e assegura o registro, evitando interrupções.

Esse mecanismo minimizou significativamente os erros de destinação de ativos, isentando os técnicos da necessidade de memorizar nomenclaturas exatas.

3.5. Exemplo de Funcionamento

Para ilustrar, considere a seguinte tarefa:

Ação: (X) Inserir

Status do ativo: (X) Em estoque

Localização do ativo: Lab Info 401

Tipo	Patrimônio	
-----	-----	
Computador	12345	

O robô:

1. Identifica que deve *inserir* o item, com *status Em estoque* e *local Lab Info 401*.
2. Converte “Lab Info 401” para a localização oficial cadastrada no GLPI (via fuzzy matching).
3. Localiza o computador de patrimônio 12345 no banco de ativos.
4. Atualiza seu status e localização, e associa o item ao chamado.
5. Gera um relatório CSV e anexa ao chamado, com uma mensagem de conclusão para o técnico.

Todo o processo ocorre em segundo plano, sem intervenção manual adicional.

4. Resultados e Discussões

A versão inicial entregue do ITXAutoNTI contempla um conjunto de módulos funcionais que, em conjunto, automatizam o ciclo completo de tratamento de tarefas que envolvem a movimentação de patrimônios no GLPI. Na prática, já operam as rotinas de varredura de chamados, interpretação de tarefas, execução via API para atualizar ativos, associação em lote e geração de relatórios CSV.

4.1. Entregas da Fase Alpha

Durante a Fase *Alpha*, foram consolidadas as seguintes entregas principais, testadas em ambiente de homologação e integradas em um fluxo *end-to-end*:

- **Pipeline completo de automação:** Da criação da tarefa padronizada ao encerramento com log e relatório CSV, eliminando cliques manuais por ativo.
- **Conjunto de módulos estáveis:** Cada módulo tem sua responsabilidade única e bem definida, implementado com tratamento de erros robusto e logs de execução para facilitar o *debugging* e a auditoria.
- **Mecanismo de correspondência aproximada para localização:** Através de um algoritmo de *fuzzy matching*, que permite mapear variações na nomenclatura de ambientes para registros oficiais no GLPI, reduzindo falhas causadas por inconsistência de digitação e banco de dados.
- **Documentação técnica e de usuário:** O projeto inclui um guia de setup, arquivo README com fluxos de trabalho Git e manual de passo a passo para a criação de tarefas que acionam a automação.

4.2. Impacto Mensurável e Ganhos Operacionais

Para quantificar os ganhos, comparamos três métodos de alocação e movimentação de patrimônios:

- **Manual:** Técnico opera diretamente na interface web do GLPI.
- **Script (Macro):** Solução anterior baseada em automação de interface (*AutoHot-key*), que exige preparo manual de um arquivo CSV.
- **ITXAutoNTI:** Automação via API com *fuzzy matching* e processamento em lote.

Foram simuladas movimentações de 1, 10, 20 e 200 patrimônios da mesma categoria. A Tabela 1 resume os resultados dessas medidas.

Tabela 1. Comparativo de tempo total e do número de interações para movimentação de patrimônios (inserção + alteração de status/localização)

Método	Métrica	1 item	10 itens	20 itens	200 itens
Manual	Tempo total:	34s	5min 15s	11min 49s	~1h 58min
	Cliques ativos:	7	70	140	1.400
Script (macro)	Tempo total:	38s	5min 39s	12min 07s	~2h 01min
	Cliques ativos:	2	2	2	2
ITXAutoNTI	Tempo total:	51s	55s	1min 05s	~3min 00s
	Cliques ativos:	3	3	3	3

4.3. Análise dos Resultados

- **Ganho de escala:** Para um único item, o ITXAutoNTI é ligeiramente mais lento que a execução manual devido ao tempo fixo de preparação da tarefa padronizada. Com o crescimento do lote, o ganho é expressivo: em 20 itens, o sistema é 10 vezes mais rápido; em 200 itens, o tempo cai de 2 horas para 3 minutos (redução de 97,5%).
- **Redução de interações:** O método manual exige 7 cliques por item. O ITXAutoNTI reduz isso para apenas 3 interações totais, independentemente do tamanho do lote. Isso libera o técnico para atividades mais estratégicas.
- **Comparação com a macro anterior:** A macro reduz cliques, mas exige preparar manualmente um CSV (aproximadamente 10 s/ item), é frágil a mudanças da interface e bloqueia o computador; o ITXAutoNTI evita esses problemas por operar via API. Além disso, o computador não pode ser usado enquanto a macro estiver operando, sendo uma desvantagem da utilização deste método.

4.4. Impacto Operacional Consolidado

A combinação de processamento em lote via API com correção inteligente de localizações resultou em:

- Redução do tempo médio de atendimento de chamados patrimoniais em **mais de 90%** para lotes típicos (10 a 20 itens).
- Priorização do enfoque da equipe técnica para atividades analíticas e estratégicas.

4.5. Comparação Gráfica dos Resultados

- Figura 2: tempo por itens — métodos manual e por script aumentam similarmente por item; ITXAutoNTI é estável e geralmente mais rápido.
- Figura 3: cliques por itens — manual aumenta proporcionalmente; ITXAutoNTI e script mantêm cliques constantes.

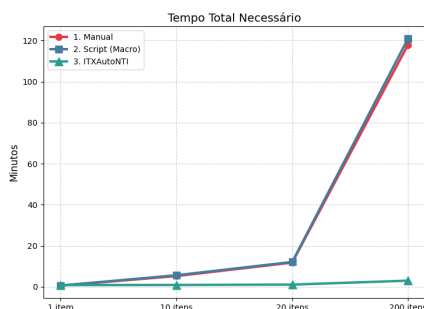


Figura 2. Tempo de alocação por número de patrimônios.

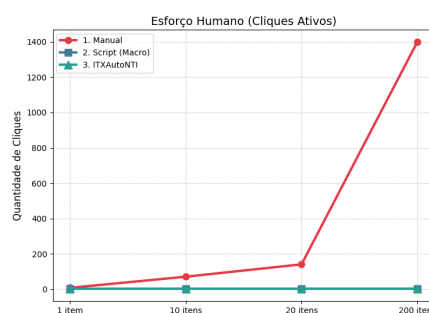


Figura 3. Cliques por alocação por número de patrimônios.

Com base nas interpretações gráficas, as características dos métodos são:

- **Manual:** O método manual tende a gastar cada vez mais tempo (Figura 2) e, segundo o número de cliques, exige atividade contínua do usuário (Figura 3).
- **Script:** O script tende a gastar cada vez mais tempo (Figura 2) e, segundo o número de cliques, exige pouca atividade contínua do usuário (Figura 3).
- **ITXAutoNTI:** O ITXAutoNTI tende a gastar pouco tempo, independentemente da quantidade de alocações (Figura 2), e, segundo o número de cliques, exige pouca atividade contínua do usuário (Figura 3).

5. Considerações Finais

A vinculação manual de ativos a chamados era repetitiva e sujeita a erros, consumindo tempo técnico. A implantação do ITXAutoNTI automatizou esse fluxo, executando em minutos tarefas antes manuais e com alta precisão.

5.1. Lições Aprendidas na Integração

O percurso de desenvolvimento evidenciou desafios técnicos e de segurança, especialmente durante a transição para a versão de produção. Destacam-se três lições aprendidas fundamentais e como foram superadas para a replicação institucional:

1. **Descoberta de Limites Nativos via JSON:** Durante a análise dos dados estruturados em JSON retornados pela API, a equipe descobriu que chamados com mais de 50 tarefas não eram processados integralmente. Para solucionar a quebra desse limite e desenvolver a paginação dinâmica de forma autônoma (sem causar gargalos), a equipe utilizou como referência arquitetural a lógica do repositório aberto *python-glpi-utils*, adaptando-a nativamente para o projeto.
2. **Modernização e Estabilidade de Requisições:** Para prevenir quedas de conexão originadas por latência do servidor, a biblioteca HTTP nativa (*requests* e *urllib*) foi preterida em favor da biblioteca *httpx*. Essa transição agregou o envio de *headers* de anulação de *cache*, consolidando uma comunicação imutável e performática com a API.
3. **Gestão de Vulnerabilidades em Módulos Analíticos:** A implementação do motor de correspondência revelou que a biblioteca inicial (*fuzzywuzzy*, baseada em *Levenshtein*) sofria com vulnerabilidades (CVEs) e obsolescência. O problema foi sanado com a reescrita do motor lógico para consumir o pacote moderno *rapidfuzz*, que provou ter maior segurança e desempenho.

A adoção do fuzzy matching mitigou associações incorretas causadas pela heterogeneidade dos dados legados e por falhas de digitação dos usuários.

5.2. Dificuldades Técnicas Durante o Desenvolvimento

A autenticação da API e a estruturação do código foram desafios iniciais por falta de experiência, mas foram superados via pesquisa e mentoria. Por outro lado, problemas técnicos mais complexos exigiram investigação aprofundada e planejamento, dentre eles, destacam-se:

1. **Diferenças dos IDs utilizados dentro e fora do GLPI:** No início, notou-se que IDs internos do GLPI não coincidiam com os números de patrimônio e que as localizações divergiam de outros sistemas; esses IDs eram necessários para atualizações via API. Contornou-se o problema com funções que via requisições HTTP e JSON retornam o ID interno do GLPI e com fuzzy matching para mapear entradas de localizações à tabela interna ligada ao banco do GLPI.
2. **Implementação do *Fuzzy Matching*:** A principal dificuldade surgiu no tratamento dos dados de entrada, definiu-se a utilização da função *token_set_ratio* da biblioteca *rapidfuzz* para este fim. O método se mostrou o mais eficiente por calcular a proximidade entre sentenças com base na interseção de palavras idênticas, ignorando a ordem ou repetições, funcionando como uma busca por palavras-chave. Para lidar com localizações de sistemas internos que não constam no GLPI, instituiu-se uma localização genérica padrão para classificar os itens que não atingiram o score mínimo de proximidade no algoritmo.

5.3. Trabalhos Futuros e Disponibilização

Superados os desafios de implementação e confirmada a otimização dos processos internos da CGP, a solução demonstra alto potencial de aplicabilidade em outras IFES que utilizam o GLPI. Para maximizar esse potencial de replicação, o código-fonte do projeto ITXAutoNTI, juntamente com sua documentação técnica completa para configuração e manutenção, foi disponibilizado publicamente em um repositório no *GitHub* [11].

Como trabalhos futuros, propõe-se a expansão das funcionalidades da ferramenta para atender a outros setores da universidade. Adicionalmente, sugere-se a integração de módulos de coleta de dados e catalogação de chamados para a geração de painéis analíticos (*dashboards*) com métricas de desempenho dos serviços.

Agradecimentos

Os autores agradecem especialmente a **Mailsom José de Souza**, pelo acompanhamento técnico e orientações práticas que contribuíram decisivamente para a solução. Reconhecemos ainda o apoio de **Sergio Roberto Meneses Carvalho**, que, com sua visão de gestão, guiou e incentivou o desenvolvimento da ferramenta. Por fim, registramos nossa gratidão a **Gislaine Cristina Ceregatti Bonilha Pinto**, pela valiosa ajuda na execução e redação deste artigo. A todos, nosso muito obrigado pelo comprometimento e pela confiança no trabalho da equipe.

Referências

- [1] Axelos. (2019). *ITIL Foundation: ITIL 4 Edition*. TSO (The Stationery Office).
- [2] Teclib' Edition. (2024). *GLPI Developer Documentation - REST API*. Disponível em: <https://glpi-developer-documentation.readthedocs.io/en/master/devapi/index.html>. Acesso em: 06 abr. 2026.
- [3] Masse, M. (2011). *REST API Design Rulebook: Designing Consistent RESTful Web Service Interfaces*. O'Reilly Media, Inc.
- [4] Python Software Foundation. (2024). *Python Language Reference, version 3.x*. Disponível em: <https://www.python.org>. Acesso em: 06 abr. 2026.
- [5] Encode OSS. (2024). *HTTPX: A next generation HTTP client for Python*. Documentação Oficial. Disponível em: <https://www.python-httpx.org/>. Acesso em: 06 abr. 2026.
- [6] Navarro, G. (2001). A guided tour to approximate string matching. *ACM Computing Surveys (CSUR)*, 33(1), 31-88.
- [7] Bachmann, M. (2026). *RapidFuzz: Rapid fuzzy string matching*. Repositório GitHub. Disponível em: <https://github.com/rapidfuzz/RapidFuzz>. Acesso em: 03 jun. 2026.
- [8] Driessen, V. (2010). *A successful Git branching model*. Disponível em: <https://nvie.com/posts/a-successful-git-branching-model/>. Acesso em: 06 abr. 2026.
- [9] Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. Scrum.org. Disponível em: <https://scrumguides.org>. Acesso em: 06 abr. 2026.
- [10] Giovanni07. (2024). *python-glpi-utils*. Repositório GitHub. Disponível em: <https://github.com/giovanny07/python-glpi-utils/tree/main>. Acesso em: 03 jun. 2026.
- [11] Equipe ITXAutoNTI. (2026). *ITXAutoNTI: Automação Inteligente de Tarefas no GLPI*. Repositório GitHub. Disponível em: <https://github.com/ITX-Duda/ITXAutoNTI>. Acesso em: 03 jun. 2026.